

Next-Gen CAPTCHAs: Leveraging the Cognitive Gap for Scalable and Diverse GUI-Agent Defense

Jiacheng Liu^{*12} Yaxin Luo^{*12} Jiacheng Cui¹² Xinyi Shang¹²³ Xiaohan Zhao¹² Zhiqiang Shen¹²

Abstract

The rapid evolution of GUI-enabled agents has rendered traditional CAPTCHAs obsolete. While previous benchmarks like OpenCaptchaWorld established a baseline for evaluating multimodal agents, recent advancements in reasoning-heavy models, such as Gemini3-Pro-High and GPT-5.2-Xhigh have effectively collapsed this security barrier, achieving pass rates as high as 90% on complex logic puzzles like “Bingo”. In response, we introduce Next-Gen CAPTCHAs, a scalable defense framework designed to secure the next-generation web against the advanced agents. Unlike static datasets, our benchmark is built upon a robust data generation pipeline, allowing for large-scale and easily scalable evaluations, notably, for backend-supported types, our system is capable of generating effectively unbounded CAPTCHA instances. We exploit the persistent human-agent “Cognitive Gap” in interactive perception, memory, decision-making, and action. By engineering dynamic tasks that require adaptive intuition rather than granular planning, we re-establish a robust distinction between biological users and artificial agents, offering a scalable and diverse defense mechanism for the agentic era.¹

1. Introduction

CAPTCHAs (Completely Automated Public Turing test to tell Computers and Humans Apart) (von Ahn et al., 2003) have long served as a lightweight, widely deployed line of defense for the open web, limiting automated abuse such

^{*}Equal contribution ¹Mohamed bin Zayed University of Artificial Intelligence (MBZUAI) ²MetaAgentX ³University College London. Correspondence to: Zhiqiang Shen <Zhiqiang.shen@mbzuai.ac.ae>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹Project page is available at https://greenoso.github.io/NextGen-CAPTCHAs_webpage/.

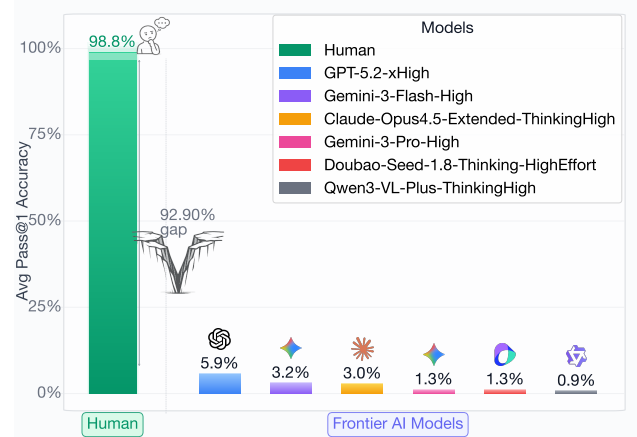


Figure 1. Frontier Models as GUI Agent Backbones’ Pass@1 on our Next-Gen CAPTCHA benchmark.

as credential stuffing, scraping, fake account creation, and transaction fraud. Historically, CAPTCHA design has followed an arms race (Yan & Ahmad, 2009; Bursztein et al., 2014; Tariq et al., 2023): early distorted-text challenges aimed to resist OCR, later image-grid CAPTCHAs targeted object recognition, and more recent “logic” CAPTCHAs introduced game-like spatial reasoning. In each cycle, progress in machine perception and reasoning steadily eroded the security margin of what was once considered “bot-hard”, forcing defenders to continuously redesign challenges and patch deployed systems.

This erosion has accelerated sharply with the emergence of Multimodal Large Language Models (MLLMs) and *GUI-enabled / Computer-Use* agents. Modern web agents can perceive rendered pages, interpret instructions, and execute multi-step interactions—capabilities demonstrated on realistic benchmarks such as Mind2Web and WebArena (Deng et al., 2023; Zhou et al., 2023). Recent security analyses further indicate that these agentic capabilities generalize zero-shot to diverse CAPTCHA challenges, effectively neutralizing the long-standing bot-hard assumption that underpins modern web security (Teoh et al., 2025). The risk is no longer theoretical: as computer-use agents become integrated into consumer and enterprise products, attackers gain an increasingly accessible “automation substrate” that can operate directly through the same browser surfaces intended for humans, widening the attack surface of websites that

rely on current CAPTCHAs as a primary gatekeeper.

At the same time, substantial capability gaps remain between humans and current MLLM-based agents, especially in *interactive* settings. Beyond static vision–language benchmarks, agents must repeatedly (i) ground instructions to precise screen regions, (ii) maintain and update a latent task state over time, and (iii) execute low-level actions robustly under partial observability and UI stochasticity. Prior work reports persistent deficits in visual–spatial grounding and intermediate state manipulation in MLLMs, and shows that offline benchmark success often degrades when models are deployed as interactive web agents in live environments (Cao et al., 2024; Yang et al., 2025; Xue et al., 2025). These gaps suggest an opportunity for defense: rather than “hardening” existing decomposable puzzles, we can design challenges that are trivial for human intuition but systematically misaligned with the over-segmented, step-by-step strategies used by today’s agents.

Motivated by this, we introduce Next-Gen CAPTCHAs: a scalable GUI-agent defense framework that produces GUI agent-defensive yet human-solvable interactive challenges. The core of Next-Gen is an automatic CAPTCHA generation pipeline with rule-based verifiable answer: for generative task families, the system can synthesize an effectively unbounded number of unique instances (e.g., timed interaction tasks such as Red Dot), each paired with a rule-based solution—eliminating the need for human annotation while maintaining strong instance diversity. Importantly, our **27 types of CAPTCHA families are newly designed by us** specifically to defend against modern GUI agents (rather than collected from existing deployed CAPTCHA systems). Except for two vision–language families, instances are procedurally generated with automatically verifiable solutions.

To make progress measurable and reproducible, we curate a *benchmark sample from* this defense system: a main test set of 519 vision–language puzzles spanning the 27 vision–language families, plus a lightweight subset with 5 puzzles per task for cost-effective evaluation under limited budgets. Under realistic closed-API agent settings on live web tasks, we observe a large human–agent gap: humans achieve near-ceiling solve rates with low latency, whereas representative high-reasoning MLLMs and GUI web agents exhibit single-digit Pass@1 (Fig. 1). We further validate human friendliness via a small-scale human study, reporting high success rates and low completion times across representative tasks.

Finally, we release a real-web evaluation platform that is agnostic of GUI framework: any GUI-enabled MLLM agent can be evaluated via a standardized browser interaction and logging interface. In our experiments, we use Browser-Use as the primary reference integration, and additionally evaluate supplementary agent frameworks including Claude Cowork and CrewAI. While the cu-

rated benchmark facilitates standardized comparison, it is best viewed as a *byproduct* of the broader Next-Gen defense system: the dataset is sampled for benchmarking, whereas the primary goal is a deployable, continuously generative CAPTCHA defense mechanism for the agentic era.

Our contributions are threefold: (1) We design 27 *new* formats of Next-Gen CAPTCHA families that target empirically observed human–MLLM gaps, aiming to be agent-defensive while remaining human-friendly. (2) We propose a procedural generation and automatic verification framework, enabling scalable deployment with effective diversity and controllable difficulty. (3) We release an open-source real-web evaluation platform and a benchmark *sampled from the defense system*, including both a 519-puzzle main set and a cost-aware subset, and evaluate GUI Agents across perception, memory, reasoning, and action under realistic interaction constraints.

2. Background

CAPTCHA systems have long been shaped by an *arms race* with AI (von Ahn et al., 2003; Bursztein et al., 2014). We summarize this progression across three capability shifts: *Perception*, *Reasoning*, and *Agency*, where defenses were eventually broken by the very models they aimed to stop.

The Era of Visual Perception. The defense against automated bots initially relied on distorted text to defeat OCR, premised on the belief that decoding warped visuals was uniquely human (von Ahn et al., 2003; Shet, 2014). However, Convolutional Neural Networks (CNNs) rendered this obsolete by solving text challenges with superhuman accuracy (Mori & Malik, 2003; Shet, 2014; Gao et al., 2016). Consequently, security mechanisms shifted to object classification (Google for Developers, 2024), assuming machines lacked the contextual grounding to identify objects in diverse scenes. Yet, the emergence of Vision Transformers (Dosovitskiy et al., 2021) and large-scale pre-training has since bridged this gap. Modern backbones can now interpret complex scenes with precision (Plesner et al., 2024; Sivakorn et al., 2016; Hossen et al., 2020), effectively neutralizing static visual perception as a reliable security barrier.

The Era of Multimodal Logic: Reasoning vs. MLLMs.

To counter advanced vision models, providers like Arkose Labs introduced “Logic CAPTCHAs” (Arkose Labs, 2025) puzzles requiring not just recognition, but spatial reasoning and game-like logic (e.g., rotating objects or matching icons). Until recently, these were considered secure against standard vision models. However, the emergence of Multimodal Large Language Models (MLLMs) like GPT-5.2 (OpenAI, 2025a), Claude4.5-Opus (Anthropic, 2025) and Gemini3-Pro (Google, 2025) changed the landscape. Recent benchmarks, such as MCA-Bench (Wu et al., 2025)

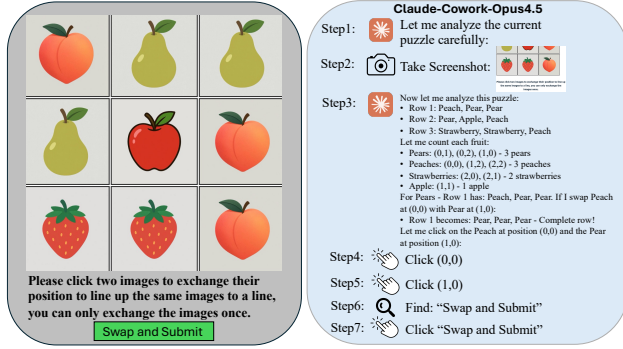


Figure 2. With the enhanced Computer Use abilities like taking screenshot, clicking and dragging etc, along with advanced thinking and tool-use power (e.g. searching), Claude-Cowork-Opus4.5 can now solve “Bingo” CAPTCHA effectively and efficiently.

and COGNITION (Wang et al., 2025), revealed that MLLMs could effectively interpret instructions and solve these logic puzzles. Crucially, as reasoning-enhanced models continue to scale, even complex logic tasks like “Bingo” is seeing pass rates exceed 90%. In a security context, a pass rate of even 50% is a critical failure; for robust defense, machine success rates must be driven down to near 0%, as a single successful breach allows an agent to access sensitive data.

The Era of Agentic AI: The Unaddressed Threat of “GUI Agents”. We are now entering a fourth, unprecedented era: the age of Autonomous Web Agents equipped with Computer Use and GUI interaction capabilities (Anthropic, 2024; OpenAI, 2025b; Anthropic, 2026). Unlike passive solvers, these agents can plan, navigate, and execute multi-step workflows (Xue et al., 2025). Existing literature primarily focused on evaluating agent capability rather than defending against it (Luo et al., 2025; Bhardwaj et al., 2026; Zhang et al., 2025; Kharlamova et al., 2025). Although some works attempt to design individual captchas (Ding et al., 2025), there are currently no systematic defenses designed specifically to prevent these agents. Current CAPTCHAs fail to differentiate between a human user and an agent that can “think” through a workflow (Deng et al., 2025; Qi et al., 2026). This creates a critical vulnerability: existing defenses are now merely speed bumps for agents that can reason as effectively as humans (Teoh et al., 2025).

This paper addresses this specific void. We argue that defending against the next generation of agents requires moving beyond static difficulty to exploiting the “Cognitive Gap”, designing tasks that are trivial for human intuition but computationally exhaustive for the over-planning nature of agentic reasoning.

3. Advanced GUI Agents Break Current CAPTCHA System

As shown in Fig. 3, the security barrier provided by many current CAPTCHA systems has effectively collapsed under

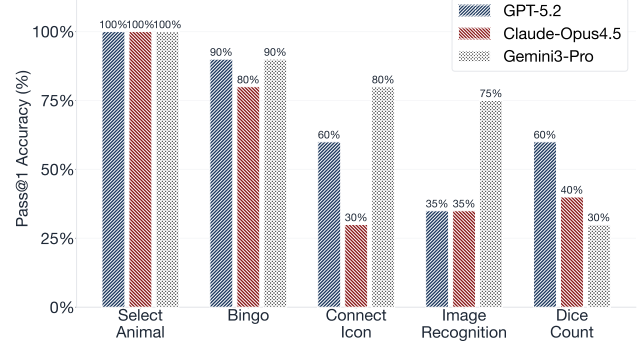


Figure 3. **Current CAPTCHAs System Fails.** Recent Advanced MLLMs can even break the current CAPTCHAs system without extra high thinking efforts (We use the default thinking settings from their official API)

state-of-the-art MLLMs powered GUI Agents. On traditional logic puzzles (e.g., *Select Animal*), leading models such as Gemini3-Pro and GPT-5.2 reach 100% Pass@1. Even on more complex reasoning tasks like *Bingo*, which requires cross-referencing visual evidence with symbolic rules, models achieve 80–90% Pass@1. Overall, *static logic is no longer a bot-hard problem*.

Perception–Thinking–Action Analysis of GUI Agents.

To understand why advanced GUI agents solve prior CAPTCHA designs so effectively, we analyze Claude-Cowork-Opus4.5 (Fig. 2). The core issue is structural: most current CAPTCHAs are short, decomposable workflows. Once agent obtains an observation of the page state, the remaining problem becomes a sequence of locally verifiable micro-decisions followed by straightforward UI operations, a regime where modern tool-using agents are highly reliable. We illustrate this with the *Bingo* CAPTCHA in Fig. 2. The agent (i) screenshots and extracts a structured grid state, (ii) enumerates candidate swaps under the rule, and (iii) executes two clicks and submits. A representative fragment is: Take Screenshot → parse grid into structured state (“Row 1: ...”) → Find a swap that completes a row → click two cells → “Swap and Submit”

This highlights a key point: once a CAPTCHA reduces to a small search over a discrete state space (grid states, icon matches, counting outcomes, or arithmetic choices), an agent with accurate perception and reliable interaction can solve it consistently because each step can be checked locally and the interface provides immediate feedback.

Correlation Analysis Reveals a Structural Difference from Current CAPTCHAs.

Beyond qualitative traces, we quantify how success relates to trajectory behavior by computing correlations between each CAPTCHA family’s Pass@1 and logged interaction/reasoning metrics. Fig. 4 contrasts widely used web-service CAPTCHAs (Luo et al., 2025) (e.g., *Bingo*, *Dice Count*) with our Next-Gen families, evaluated using a Browser-Use agent powered by Gemini3-

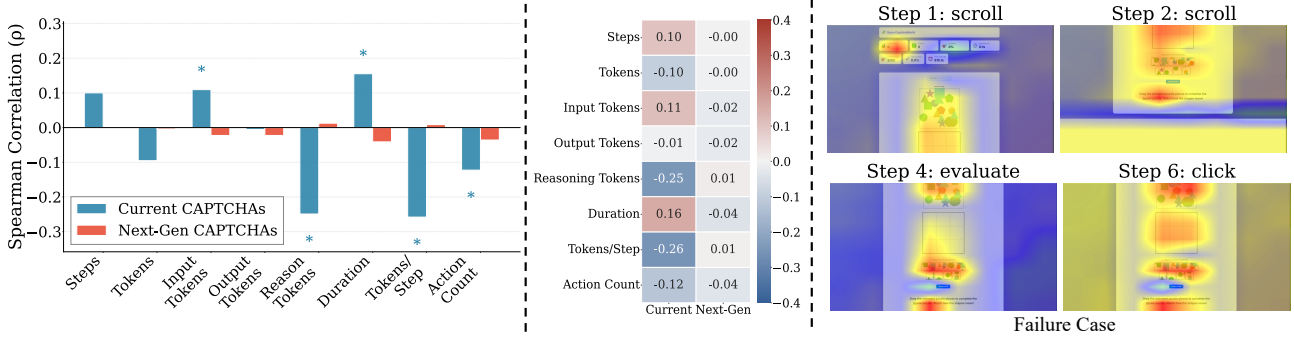


Figure 4. **Success–trajectory correlation differs between current and Next-Gen CAPTCHAs.** Left & Middle: Spearman ρ between each CAPTCHA family’s Pass@1 and logged trajectory metrics (bar plot on the left; heatmap on the right; * denotes significance). Current CAPTCHAs show non-trivial correlations, while Next-Gen correlations are near zero. Right: a representative Next-Gen failure on a jigsaw-like puzzle, where the agent scrolls/evaluates but skips drag-and-drop and prematurely clicks Submit.

Flash-High. For current CAPTCHAs, Pass@1 shows non-trivial correlations: success is weakly positively correlated with interaction length (e.g., Steps, Duration), while being negatively correlated with reasoning-heavy signals (e.g., Reasoning Tokens, Tokens-per-Step). This pattern is consistent with existing CAPTCHAs being *workflow-friendly*: once the page state is observed, solving reduces to short, decomposable micro-decisions and UI actions, and extra deliberation often reflects uncertainty rather than progress.

For Next-Gen CAPTCHAs, correlations stay near zero across metrics. More steps or a larger deliberation budget rarely improve success. Fig. 4 (right) shows why: the agent may attend to the right region and still choose the wrong *action primitive*. In the jigsaw-like CAPTCHA, solving requires drag-and-drop. The agent never drags and clicks Submit at Step 6. These failures come from interactive bottlenecks: misreading UI affordances, weak grounding for manipulation targets, or premature termination. Extra textual reasoning does not fix them. Taken together, stronger multimodal perception plus computer-use already breaks many current CAPTCHAs by following decomposable workflows. We therefore avoid making the same tasks “harder”. Instead, we design CAPTCHAs that exploit the Cognitive Gap: they require robust interactive grounding and manipulation under realistic interaction constraints.

4. Next-Gen CAPTCHAs: Native GUI-Agent Era’s Security Defense

4.1. Agent-CAPTCHA Interaction as Extended POMDP

Solving a CAPTCHA as a GUI agent is an interactive process: the solver repeatedly observes a live webpage under partial information, maintains an internal task state, performs reasoning to make decision, and executes low-level browser actions. Our design goal is to exploit a persistent human–agent gap in this observation–memory–decision–action loop: humans can often infer

the correct latent task state and act reliably with effective deliberation, whereas modern GUI agents frequently fail due to brittleness in perception, state maintenance, and perception-to-action alignment under realistic interaction constraints. We use an extended POMDP formulation to make this loop explicit and to organize the failure modes that Next-Gen CAPTCHAs intentionally amplify.

We model the GUI-agent as an extended POMDP:

$$\mathcal{W} = (S, O, X, A_{\text{web}}, A_{\text{think}}, Z, T_{\text{env}}, U, R, \kappa), \quad (1)$$

At step t , the live webpage has state $s_t \in S$, the agent receives an observation $o_t \sim Z(\cdot \mid s_t)$, maintains an internal workspace $x_t \in X$, and selects $a_t = (a_t^{\text{web}}, a_t^{\text{think}}) \in A_{\text{web}} \times A_{\text{think}}$, where a_t^{web} are browser actions (click/scroll/type/drag) and a_t^{think} is internal deliberation (e.g., a backbone thinking mode). The webpage evolves as $s_{t+1} \sim T_{\text{env}}(\cdot \mid s_t, a_t^{\text{web}})$, and the agent updates its workspace by

$$x_{t+1} = U(x_t, o_t, a_t^{\text{think}}), \quad (2)$$

We view U as comprising an observation-to-memory write and an optional deliberation step; when no extra deliberation is invoked ($a_t^{\text{think}} = \emptyset$), U reduces to a memory-only update.

Each CAPTCHA instance induces an initial state distribution ρ over S and a short horizon H (termination when the verifier returns pass/fail). The verifier provides a terminal reward $R \in \{0, 1\}$ and we measure deliberation cost $c_t = \kappa(a_t^{\text{think}})$ (tokens/api cost).

Instantiation in our main experiments. Although $\mathcal{W}$ is a general formulation applicable to multiple GUI-agent stacks, our main results use browser-use as the reference integration and restrict the observation payload to what is actually exposed at test time. Concretely, we use

$$o_t = (I_t, D_t, \text{meta}_t), \quad (3)$$

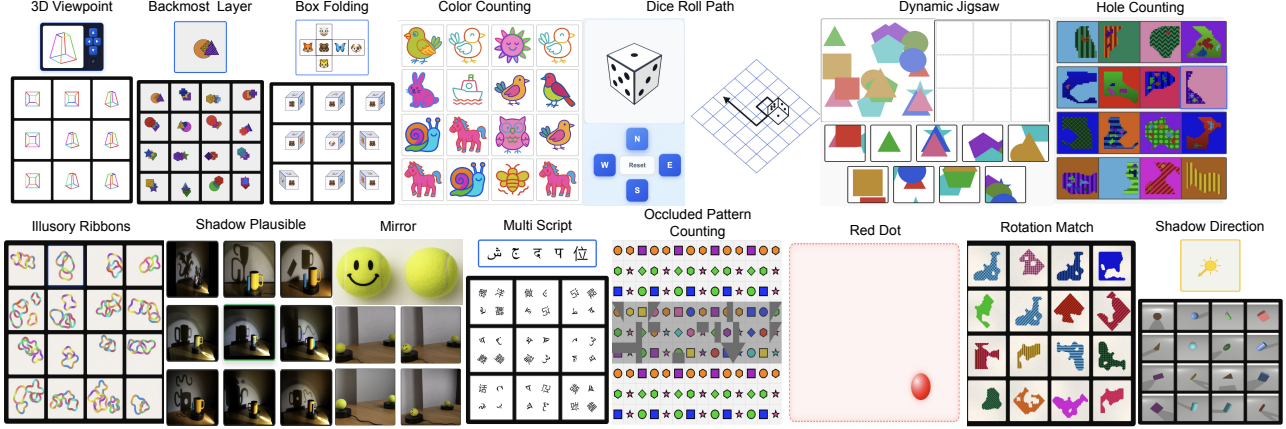


Figure 5. **Next-Gen CAPTCHA family examples.** Representative instances from Next-Gen CAPTCHA task families. The full family list is in Table 7; additional examples for all families are in Appendix E.1.

where I_t is a screenshot, meta_t includes URL and view-port/page statistics, and D_t is a filtered, DOM-derived interaction view (indexed interactive elements and puzzle-specific fields such as `window.currentPuzzle`), not the full DOM tree. We do not provide additional overlay/SoM annotation channels in the observation.

Human-agent cognitive decision gaps. Our goal is the standard CAPTCHA goal: easy for humans, hard for GUI agents. We start from general, recurring human-agent gaps reported in prior works and observations (Upadhyay et al., 2025), and we turn them into concrete CAPTCHA families. In the POMDP view, this means designing instances where success depends on reliably extracting task-relevant information from observations o_t , maintaining the correct task state x_t across steps, and executing the appropriate browser action a_t^{web} . We then instantiate these general gaps under our evaluation setting $\mathcal{W}$ (i.e., the observation payload and action interface exposed at test time), which yields the targeted gap categories below.

(G1) Scene-structure inference (observation interpretation and grounding). We target failure modes where an agent cannot reliably extract the task-relevant structure of the page from its observations under partial observability, e.g., mis-parsing layered depth cues, reflections, or shadow geometry, or grounding the instruction to the wrong visual entity (Pothiraj et al., 2025; Liu et al., 2025; Lee et al., 2025; Motamed et al., 2025). In $\mathcal{W}$, this corresponds to brittleness of *observation interpretation* under the observation channel $Z(\cdot | s_t)$: even when the underlying page state is fixed, small ambiguities or nuisance factors in o_t lead to inconsistent task-relevant features, yielding an incorrect internal decision state x_t and downstream actions.

(G2) Temporal integration (multi-step evidence accumulation). We include tasks where the decisive information is

not contained in any single snapshot, but is revealed only through interaction over multiple steps (e.g., motion cues or sequential reveals) (Upadhyay et al., 2025; Bordes et al., 2025; Yuan et al., 2025). In $\mathcal{W}$, success requires using the workspace update $x_{t+1} = U(x_t, o_t, a_t^{\text{think}})$ to integrate evidence across steps, rather than reacting myopically to a single o_t . These tasks therefore stress whether the agent can stably integrate partial evidence across time into x_t .

(G3) Numerosity and discrete invariants (decision-boundary sensitivity). We include tasks whose correct answer depends on discrete quantities or invariants (counts, parity, path endpoints), so that small perceptual errors can flip the final decision (Guo et al., 2025; Weng et al., 2025; Tamarapalli et al., 2025). In $\mathcal{W}$, this manifests as high sensitivity of the verifier reward R to discrete task variables $g(s_t)$: small misreadings of o_t can change the inferred value of $g(s_t)$ and thereby switch the rewarded outcome.

(G4) Latent-state tracking (working-memory consistency). We include tasks that require carrying intermediate variables across steps, such as partial counts, orientations, or rule states, that may not be fully re-observable later (Zhang et al., 2024; Huang et al., 2026). In $\mathcal{W}$, this corresponds to maintaining these intermediates within x_t and updating them consistently via the memory dynamics,

$$x_{t+1} = U(x_t, o_t, \emptyset). \quad (4)$$

so that subsequent decisions and interactions remain coherent over the episode.

(G5) Perception-to-action alignment (robust low-level execution). Distinct from perceptual understanding, we stress whether an agent can reliably translate a correct internal decision into the correct browser interaction (Cheng et al., 2024; Li et al., 2025). In $\mathcal{W}$, this is sensitivity in selecting and executing browser actions $a_t^{\text{web}} \in A_{\text{web}}$ such

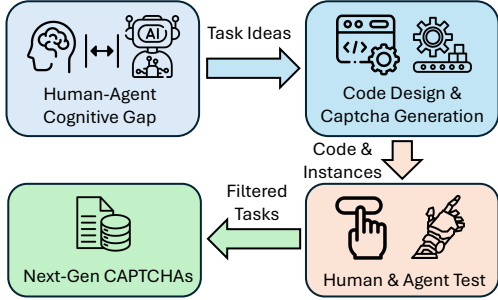


Figure 6. **Next-Gen CAPTCHA data curation pipeline.** CAPTCHA task families are first designed to target human-agent cognitive gaps, then instantiated via procedural generation with rule-based verification. A lightweight model-based stress test and human test filters families that remain difficult for GUI agents while being human-solvable, yielding a curated benchmark sampled from a continuously generative defense system.

that intended transition is realized under the environment dynamics T_{env} (e.g., precise drag targets), while remaining within practical deliberation cost κ .

4.2. Data Curation Pipeline

We build the benchmark as a curated snapshot of a larger CAPTCHA defense system. Starting from the human-agent gaps described above, we formed an initial pool of candidate CAPTCHA task families, informed by both classic CAPTCHA design and recent studies (Upadhyay et al., 2025). For each candidate task family, we implemented an end-to-end task: a web interface, an instance generator when applicable, where the puzzle rules and solution are encoded directly in the generation process, ensuring correctness by construction rather than by an external checker.

Before running any model-based filtering, we conducted a preliminary manual verification to confirm that the rule-based generation behaves as intended and that the generated puzzles are interpretable and human-friendly and the task is genuinely usable as a CAPTCHA: instructions are unambiguous, interactions work as intended, and typical users can solve without domain knowledge. We then produced a small pilot set for each task family: 20 generated instances for generative families, and comparable curated examples for the few non-generated ones.

To decide which families are worth keeping, we used a lightweight but strong model (Gemini 3 Flash) as a first-pass stress test. We retained a family only when it remained difficult for the model (Pass@1 below 30% on the pilot set) while being easy for humans (above 90% success on a randomly sampled subset of 10 pilot instances). This filtering yielded 27 vision-language CAPTCHA families. Among them, 25 families are fully script-generated with automatic checking, while 2 utilize curated human-crafted instances. From the retained families, we construct a main test set of 519 puzzles for evaluation. We also provide

a budget-friendly subset by sampling 5 puzzles per family (135 total) to support quick comparisons under limited query budgets, while preserving coverage across task types.

Summary of Next-Gen CAPTCHAs design objectives and properties. Table 1 summarizes the key objectives and principles behind our Next-Gen CAPTCHAs: they are generative with rule-defined ground truth, cover diverse interaction bottlenecks, and remain human-friendly. These properties shift CAPTCHAs away from static, decomposable logic and toward tasks that require robust interactive grounding and manipulation.

Table 1. **Next-Gen CAPTCHA design summary (aligned with Cognitive-Gap categories).**

Key property	Summary	Gap				
		G1	G2	G3	G4	G5
Generative	Procedural generation; scalable; resists memorization.	.	.	.	.	.
Rule-defined GT	GT verified by generation rules; no manual labels.	.	.	.	.	.
Diverse coverage	Covers G1–G5 families; avoids single-strategy solvers.	•	•	•	•	•
Human-friendly	High human success without domain knowledge.	.	.	.	.	.
Agent-defensive	Stresses realistic GUI interaction; mainly G5 ⁺ .	◦	.	◦	◦	•
Cognitive-Gap targeting	Interactive bottlenecks to maximize human-agent separation.	•	•	•	•	•

Legend: • primary; ◦ secondary; . not targeted.

G1–G5 exemplars: **G1** Mirror; **G2** Structure From Motion; **G3** Hole Counting; **G4** Box Folding; **G5** Static Jigsaw. Full family catalog: see Table 7. ⁺ also stresses secondary gaps (e.g., G1/G4).

5. Experiments and Analysis

5.1. Experimental Settings

We report Pass@1 under a fixed interactive evaluation protocol on live web CAPTCHAs. Unless stated otherwise, all main results use the Browser-Use GUI-agent as the default framework, running each CAPTCHA as a real-browser episode with state reset between puzzles. At each step, the agent observes a multimodal page state (viewport screenshot + DOM-derived interactive elements + lightweight metadata) and outputs structured browser actions (e.g., click, type, drag, scroll) executed by Playwright in visible mode; success is determined by the platform’s automatic verification signals. More details are in Appendix. A.

5.2. Results on Next-Gen’s Testing Benchmark

Table 2 summarizes the performance of different MLLM backbones under the same Browser-Use baseline agent on Next-Gen’s testing benchmark. Humans achieve near-ceiling performance (98.8% Pass@1), while all evaluated MLLM-based agents are successfully defended by our CAPTCHAs, with the best-performing backbone reaching

Table 2. Performance of different MLLM backbones within the Browser-Use baseline agent on Next-Gen Captcha. Darker “■” indicates higher success rate@1 and darker “■” indicates higher cost.

Captcha Type	Human	GPT-5.2 xHigh	Gemini-3 Flash-High	Claude-Opus4.5 Extend-ThinkingHigh	Gemini-3 Pro-High	Doubao-Seed-1.8 Thinking-HighEffort	Qwen3-VL-Plus ThinkingHigh
Avg Pass@1 (%)	98.8	5.9	3.2	3.0	1.3	1.3	0.9
Total Cost (\$)	–	3122.3	6.5	224.0	50.1	26.6	28.3
3D_Viewpoint	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Backmost_Layer	100.0	20.0	0.0	0.0	0.0	0.0	0.0
Box_Folding	100.0	20.0	0.0	0.0	0.0	0.0	0.0
Color_Counting	100.0	40.0	5.0	0.0	0.0	0.0	5.0
Dice_Roll_Path	100.0	0.0	15.0	0.0	0.0	5.0	15.0
Dynamic_Jigsaw	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Hole_Counting	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Structure_From_Motion	90.0	0.0	0.0	0.0	5.0	0.0	0.0
Subway_Paths	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Temporal_Object_Continuity	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Trajectory_Recovery	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Illusory_Ribbons	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Layered_Stack	100.0	0.0	10.0	0.0	0.0	0.0	0.0
Mirror	90.0	20.0	18.2	0.0	9.1	9.1	0.0
Multi_Script	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Occluded_Pattern_Counting	100.0	20.0	0.0	0.0	5.0	15.0	0.0
Red_Dot	100.0	0.0	15.0	20.0	0.0	0.0	0.0
Rotation_Match	100.0	20.0	0.0	0.0	0.0	0.0	0.0
Shadow_Direction	100.0	20.0	0.0	0.0	0.0	0.0	0.0
Shadow_Plausible	100.0	0.0	12.5	0.0	0.0	0.0	0.0
Spooky_Circle_Grid	100.0	0.0	0.0	0.0	5.0	0.0	0.0
Spooky_Jigsaw	90.0	0.0	0.0	0.0	0.0	0.0	0.0
Spooky_Shape_Grid	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Spooky_Size	100.0	0.0	5.0	0.0	10.0	0.0	5.0
Spooky_Text	100.0	0.0	0.0	0.0	0.0	0.0	0.0
Static_Jigsaw	100.0	0.0	0.0	60.0	0.0	0.0	0.0
Spooky_Circle	100.0	0.0	5.0	0.0	0.0	5.0	0.0

only 5.9% Pass@1. This stark discrepancy indicates that Next-Gen CAPTCHAs preserve a substantial security margin against current GUI-enabled agents. We also report the cost of each MLLMs backbone on Next-Gen CAPTCHAs, with more detailed analysis in Section 5.3.

Among agent backbones, GPT-5.2-xHigh yields the highest Pass@1 (5.9%), followed by Gemini-3-Flash-High (3.2%) and Claude-Opus4.5-Extended-ThinkingHigh (3.0%). Gemini-3-Pro-High (1.3%), Doubao-Seed-1.8-Thinking-HighEffort (1.3%), and Qwen3-VL-Plus-ThinkingHigh (0.9%) cluster around $\sim 1\%$. Notably, these are widely regarded as the strongest reasoning MLLMs up-to-date, yet their success remains low, suggesting dominant errors arise from interactive deployment bottlenecks rather than perception or language reasoning alone: precise spatial grounding, maintaining intermediate state across steps, and robust action execution within the UI. Consistent with Fig. 4, agents often attend to the correct region but choose wrong *action primitive* (e.g., clicking Submit instead of dragging/rotating or taking more screenshots for further analysis), so additional steps and budget do not increase success. From a security perspective, residual Pass@1 can be further reduced by the rate limits and interaction friction, while attackers’ pay attempts and long trajectories create a favorable economic asymmetry for the defenders.

5.3. Ablation and Analysis

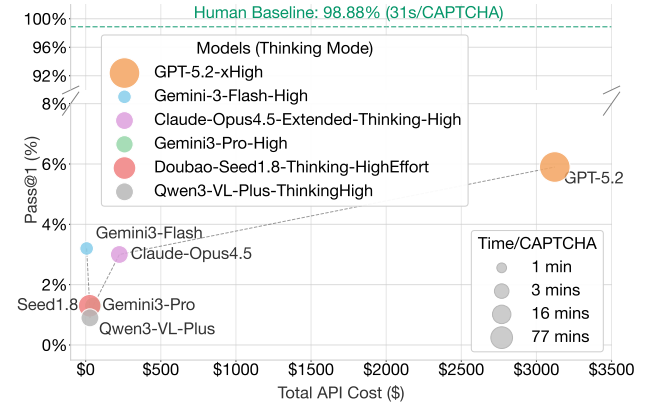


Figure 7. Cost-accuracy-latency trade-off on Next-Gen. We plot each model’s Pass@1 (y-axis) against total API cost (x-axis); bubble size indicates average time per CAPTCHA.

Cost-Efficiency and Economic Asymmetry. We evaluate the economic viability of attacking Next-Gen CAPTCHAs by analyzing the trade-off between attack success rates (Pass@1), API costs (billed usage) and time to finish the evaluation in a live API setting across varying MLLMs.

As illustrated in Fig. 7, we observe a distinct lack of correlation between computational expenditure and task success, effectively breaking the scaling laws typically seen in static

Table 3. Ablation on agent frameworks (overall). Overall Pass@1 (%) on the 135-puzzle subset of the Next-Gen CAPTCHA benchmark under different GUI-agent frameworks. Backbone LLM and protocol are held constant (Claude-Opus4.5).

Framework	Overall Pass@1 (%)
CrewAI	0.00
Browser-Use	1.48
Claude Cowork	4.44

benchmarks. While the human baseline demonstrates a near-perfect success rate of 98.8% with an average completion time of just 31 seconds, even the most advanced frontier models fail to achieve meaningful penetration. Notably, GPT-5.2-Xhigh, utilizing its maximum reasoning capacity, incurs an exorbitant cost around \$3,122 to process the evaluation set yet achieves a Pass@1 rate of only 5.9%. This represents a highly unfavorable cost–benefit for potential attackers, as scaling inference costs by orders of magnitude yields negligible gains in solvability.

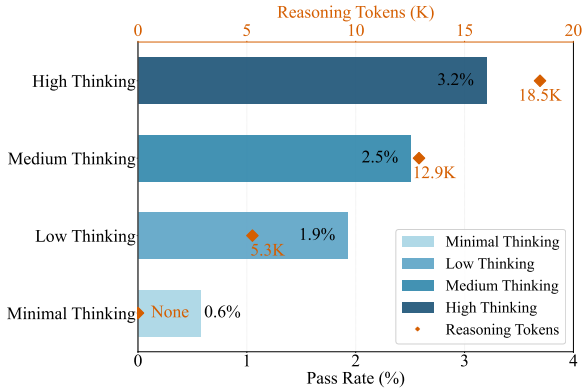


Figure 8. Thinking-mode ablation on Gemini-3-Flash. Left: Pass@1 under four thinking modes (Minimal/Low/Medium/High). Right: the corresponding average reasoning-token usage. While higher reasoning budget yields consistent gains, improvements quickly saturate and remain far from reliable success, indicating that Next-Gen CAPTCHAs are not reasoning-heavy and that interactive perception–action bottlenecks dominate failures.

Furthermore, the latency overhead introduced by agentic reasoning serves as a secondary defense layer. The bubble sizes in Fig. 7 indicate that high-reasoning models can require anywhere from 16 to 77 minutes to attempt a single complex puzzle, a timeframe that renders real-time automated attacks operationally infeasible compared to the rapid sub-minute performance of human users. This confirms that Next-Gen CAPTCHAs successfully impose a severe economic and temporal asymmetry, ensuring that automated attacks remain both technically intractable and financially ruinous regardless of the compute budget allocated.

Reasoning Marginally Helps, but Next-Gen Is Not Reasoning-Heavy. We further analyze whether allocating more deliberate reasoning budget can meaningfully improve an agent’s ability to solve Next-Gen CAPTCHAs. As shown

in Fig. 8, increasing the thinking mode of Gemini3-flash consistently improves Pass@1, indicating that reasoning is indeed helpful. However, the improvement is small and quickly saturates: Pass@1 rises only from 0.6% (Minimal) to 1.9% (Low), 2.5% (Medium), and 3.2% (High), despite a large increase in reasoning-token usage (from none to 5.3K/12.9K/18.5K tokens for Low/Medium/High). This pattern suggests that Next-Gen CAPTCHAs are *not* primarily limited by long-horizon textual reasoning; instead, failures are dominated by interactive bottlenecks such as fine-grained visual–spatial grounding, temporal evidence integration, short-term state maintenance across steps, and robust low-level action execution. Consequently, our Next-Gen families remain strongly agent-defensive even against today’s reasoning-augmented models: simply “thinking more” is insufficient to reliably pass.

Agent Frameworks Matter, but Do Not Close the Gap.

To test whether our results depend on the agent orchestration layer, we ablate the underlying GUI-agent frameworks on the 135-puzzle Next-Gen CAPTCHA subset, while holding the browser interface, evaluation protocol, and backbone LLM fixed (Claude-Opus4.5). Table 3 reports the overall Pass@1 for three representative, up-to-date frameworks. We observe a clear separation: CrewAI achieves 0.00% Pass@1, Browser-Use reaches 1.48%, and Claude Cowork attains 4.44%. These differences indicate that the interaction stack and action execution quality can measurably affect outcomes. However, the absolute performance remains uniformly low across all frameworks, showing that improved orchestration alone does not eliminate the fundamental difficulty posed by Next-Gen CAPTCHAs.

6. Conclusion

We present **Next-Gen CAPTCHAs**, a systematic defense framework designed to secure the web against the advanced autonomous GUI-enabled agents. Our investigation reveals that the arms race between CAPTCHAs and AI has reached a tipping point: recent advancements in reasoning-heavy MLLMs, such as GPT-5.2 and Gemini3-Pro, have effectively trivialized existing logic-based benchmarks, achieving near-perfect pass rates on previously secure tasks. This collapse of the bot-hard assumption exposes critical vulnerabilities in the current web infrastructure, where agents can now navigate, reason, and act with human-like proficiency. To counter this, we introduce a paradigm shift from static difficulty to dynamic cognitive divergence. By exploiting the “Cognitive Gap”, the inherent asymmetry between human intuition and the over-segmented, costly reasoning processes of current agents, we successfully restore the distinction between biological and artificial users. Moreover, our pipeline enables continual refresh at scale and provides a practical path for deploying CAPTCHA services that can defend real-world web systems against advanced GUI agents.

Impact Statement

This paper studies the security implications of modern multimodal GUI agents on existing CAPTCHA systems and proposes Next-Gen CAPTCHAs together with an evaluation platform to measure agent robustness in realistic web-interaction settings. Our goal is to advance machine learning and its safe deployment by identifying concrete failure modes of current agents, and by providing a human-friendly, scalable defense mechanism against automated abuse.

Potential benefits. The primary expected benefit is improved web security and trustworthiness: stronger agent-resistant CAPTCHAs can help mitigate automated fraud, credential stuffing, spam, mass account creation, scraping, and other forms of abuse. Our benchmark and platform can also support research on safer and more reliable interactive agents by enabling reproducible evaluation of perception-action grounding, memory, and manipulation under real-world constraints.

Potential risks and misuse. As with many security-related publications, our work could be dual-use. Detailed descriptions of CAPTCHA design and solver behavior may aid adversaries in developing more effective bypass strategies, or in stress-testing deployed defenses. To mitigate this risk, our focus is on principles for agent-defensive challenge design and on evaluation methodology rather than releasing attack tooling; moreover, procedural generation enables rapid instance diversification, which can reduce the value of static memorization-based attacks.

Fairness, accessibility, and user burden. CAPTCHAs can impose friction and may disproportionately impact users with disabilities or limited access to audio/visual modalities. We therefore emphasize human friendliness in design and include a human study to measure completion time and success rate; nevertheless, we acknowledge that accessibility and usability require ongoing attention for real deployments (e.g., alternative modalities, localization, and adaptive difficulty). Overall, we believe the potential benefits to web security and the responsible evaluation of GUI agents outweigh the foreseeable risks, and we encourage future work on accessibility-aware and privacy-preserving deployment of Next-Gen CAPTCHA defenses.

Acknowledgements

This work is supported by the MBZUAI-WIS Joint Program for Artificial Intelligence Research.

References

Anthropic. Introducing computer use, a new claude 3.5 sonnet, and claude 3.5 haiku. 2024. URL [https://www.anthropic.com/news/](https://www.anthropic.com/news/3-5-models-and-computer-use)

[3-5-models-and-computer-use](https://www.anthropic.com/news/3-5-models-and-computer-use).

Anthropic. Introducing claude opus 4.5. November 2025. URL <https://www.anthropic.com/news/claude-opus-4-5>. Accessed 2026-01-19.

Anthropic. Cowork: Claude code for the rest of your work. January 2026. URL <https://claude.com/blog/cowork-research-preview>. Accessed 2026-01-19.

Arkose Labs. Arkose matchkey advanced captcha software. 2025. URL <https://www.arkoselabs.com/arkose-matchkey/>.

Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

Bhardwaj, A., Diwan, N., and Wang, G. Beyond beautifulsoup: Benchmarking llm-powered web scraping for everyday users. *arXiv preprint arXiv:2601.06301*, 2026.

Bordes, F., Garrido, Q., Kao, J. T., Williams, A., Rabbat, M., and Dupoux, E. Intphys 2: Benchmarking intuitive physics understanding in complex synthetic environments. *arXiv preprint arXiv:2506.09849*, 2025.

browser-use. browser-use. 2026. URL <https://github.com/browser-use/browser-use>.

Bursztein, E., Aigrain, J., Moscicki, A., and Mitchell, J. C. The end is nigh: Generic solving of text-based CAPTCHAs. In *8th USENIX Workshop on Offensive Technologies (WOOT 14)*, San Diego, CA, August 2014. USENIX Association. URL <https://www.usenix.org/conference/woot14/workshop-program/presentation/bursztein>.

ByteDance Seed. Seed1.8 model card: Towards generalized real-world agency. December 2025. URL <https://lf3-static.bytednsdoc.com/obj/eden-cn/lapzild-tss/1jhwZthlaukjlkulzlp/research/Seed-1.8-Modelcard.pdf>. Accessed 2026-01-29.

Cao, X., Shen, Y., Lai, B., Ye, W., Ma, Y., Heintz, J., Chen, J., Huang, M., Cao, J., Zhang, A., et al. What is the visual cognition gap between humans and multimodal llms? *arXiv preprint arXiv:2406.10424*, 2024.

Cheng, K., Sun, Q., Chu, Y., Xu, F., YanTao, L., Zhang, J., and Wu, Z. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9313–9332, 2024.

- crewAI. crewAI. 2026. URL <https://github.com/crewAIInc/crewAI>.
- Deng, G., Ou, H., Liu, Y., Zhang, J., Zhang, T., and Liu, Y. Oedipus: Llm-enhanced reasoning captcha solver. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pp. 6–20, 2025.
- Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- Ding, Z., Deng, G., Liu, Y., Ding, J., Chen, J., Sui, Y., and Li, Y. Illusioncaptcha: A captcha based on visual illusion. In *Proceedings of the ACM on Web Conference 2025*, pp. 3683–3691, 2025.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houselby, N. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2010.11929>.
- Gao, H., Yan, J., Cao, F., Zhang, Z., Lei, L., Tang, M., Zhang, P., Zhou, X., Wang, X., and Li, J. A simple generic attack on text captchas. In *Network and Distributed System Security Symposium (NDSS)*, 2016. URL <https://www.ndss-symposium.org/wp-content/uploads/2017/09/simple-generic-attack-text-captchas.pdf>.
- Google. A new era of intelligence with gemini 3. November 2025. URL <https://blog.google/products-and-platforms/products/gemini/gemini-3/>. Accessed 2026-01-19.
- Google for Developers. recaptcha v2. 2024. URL <https://developers.google.com/recaptcha/docs/display>.
- Guo, X., Huang, Z., Shi, Z., Song, Z., and Zhang, J. Your vision-language model can’t even count to 20: Exposing the failures of vlms in compositional counting. *arXiv preprint arXiv:2510.04401*, 2025.
- Hossen, M. I., Tu, Y., Rabby, M. F., Islam, M. N., Cao, H., and Hei, X. An object detection based solver for {Google’s} image {reCAPTCHA} v2. In *23rd international symposium on research in attacks, intrusions and defenses (RAID 2020)*, pp. 269–284, 2020.
- Huang, J.-t., Sun, K., Wang, W., and Dredze, M. On the failure of latent state persistence in large language models. *arXiv preprint arXiv:2505.10571*, 2026. URL <https://arxiv.org/abs/2505.10571>.
- Kharlamova, A., He, B., Ma, C., and Liu, X. Spatial captcha: Generatively benchmarking spatial reasoning for human-machine differentiation. *arXiv preprint arXiv:2510.03863*, 2025.
- Lee, P. Y., Je, J., Park, C., Uy, M. A., Guibas, L., and Sung, M. Perspective-aware reasoning in vision-language models via mental imagery simulation. *arXiv preprint arXiv:2504.17207*, 2025.
- Li, K., Meng, Z., Lin, H., Luo, Z., Tian, Y., Ma, J., Huang, Z., and Chua, T.-S. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 8778–8786, 2025.
- Liu, Z., Gao, K., Liang, S., Xiao, B., Qiao, L., Ma, L., and Jiang, T. Beyond the visible: Benchmarking occlusion perception in multimodal large language models. *arXiv preprint arXiv:2508.04059*, 2025.
- Luo, Y., Li, Z., Liu, J., Cui, J., Zhao, X., and Shen, Z. Open captchaworld: A comprehensive web-based platform for testing and benchmarking multimodal llm agents. *arXiv preprint arXiv:2505.24878*, 2025.
- Microsoft. Playwright. <https://playwright.dev/>, 2026.
- Mori, G. and Malik, J. Recognizing objects in adversarial clutter: Breaking a visual captcha. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2003. doi: 10.1109/CVPR.2003.1211347. URL https://www2.cs.sfu.ca/~mori/research/papers/mori_cvpr03.pdf.
- Motamed, S., Culp, L., Swersky, K., Jaini, P., and Geirhos, R. Do generative video models understand physical principles? *arXiv preprint arXiv:2501.09038*, 2025.
- OpenAI. Introducing gpt-5.2. December 2025a. URL <https://openai.com/index/introducing-gpt-5-2/>. Accessed 2026-01-19.
- OpenAI. Introducing chatgpt agent: bridging research and action. 2025b. URL <https://openai.com/index/introducing-chatgpt-agent/>.
- Plesner, A., Vontobel, T., and Wattenhofer, R. Breaking recaptchav2. In *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 1047–1056. IEEE, 2024.

- Pothiraj, A., Stengel-Eskin, E., Cho, J., and Bansal, M. Capture: Evaluating spatial reasoning in vision language models via occluded object counting. *arXiv preprint arXiv:2504.15485*, 2025.
- Qi, M., He, D., Wang, Q., and Zhang, L. Viper strike: Defeating visual reasoning captchas via structured vision-language inference. *arXiv preprint arXiv:2601.06461*, 2026.
- Shet, V. Are you a robot? introducing no captcha recaptcha. *Google Security Blog*, 3:12, 2014.
- Sivakorn, S., Polakis, I., and Keromytis, A. D. I am robot: (deep) learning to break semantic image captchas. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016. URL <https://www.cs.columbia.edu/~suphannee/papers/sivakorn.europs2016.recaptcha.pdf>.
- Tamarapalli, J. S., Grover, R., Pande, N., and Yerramilli, S. Countqa: How well do mllms count in the wild? *arXiv preprint arXiv:2508.06585*, 2025.
- Tariq, N., Khan, F. A., Moqurrah, S. A., and Srivastava, G. Captcha types and breaking techniques: Design issues, challenges, and future research directions. *arXiv preprint arXiv:2307.10239*, 2023.
- Teoh, X., Lin, Y., Li, S., Liu, R., Sollomoni, A., Harel, Y., and Dong, J. S. Are {CAPTCHAs} still bot-hard? generalized visual {CAPTCHA} solving with agentic vision language model. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 3747–3766, 2025.
- Upadhyay, U., Ranjan, M., Shen, Z., and Elhoseiny, M. Time blindness: Why video-language models can’t see what humans can? *arXiv preprint arXiv:2505.24867*, 2025.
- von Ahn, L., Blum, M., Hopper, N. J., and Langford, J. Captcha: Using hard ai problems for security. In *Advances in Cryptology – EUROCRYPT 2003*, volume 2656 of *Lecture Notes in Computer Science*, pp. 294–311. Springer, 2003. doi: 10.1007/3-540-39200-9_18. URL https://link.springer.com/chapter/10.1007/3-540-39200-9_18.
- Wang, J., Zhu, C., Zhou, Y., Li, L., He, X., and Xiong, J. Cognition: From evaluation to defense against multimodal llm captcha solvers. *arXiv preprint arXiv:2512.02318*, 2025.
- Weng, T., Wang, J., Jiang, W., and Ming, Z. Visnumbench: Evaluating number sense of multimodal large language models. *arXiv preprint arXiv:2503.14939*, 2025.
- Wu, Z., Xue, Y., Feng, Y., Wang, X., and Song, Y. Mca-bench: A multimodal benchmark for evaluating captcha robustness against vlm-based attacks. *arXiv preprint arXiv:2506.05982*, 2025.
- Xue, T., Qi, W., Shi, T., Song, C. H., Gou, B., Song, D., Sun, H., and Su, Y. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382*, 2025.
- Yan, J. and Ahmad, A. S. E. CAPTCHA security: A case study. *IEEE Secur. Priv.*, 7(4):22–28, 2009. doi: 10.1109/MSP.2009.84. URL <https://doi.org/10.1109/MSP.2009.84>.
- Yang, J., Yang, S., Gupta, A. W., Han, R., Fei-Fei, L., and Xie, S. Thinking in space: How multimodal large language models see, remember, and recall spaces. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 10632–10643, 2025.
- Yuan, Y., Zhang, H., Li, W., Cheng, Z., Zhang, B., Li, L., Li, X., Zhao, D., Zhang, W., Zhuang, Y., et al. Videorefer suite: Advancing spatial-temporal object understanding with video llm. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 18970–18980, 2025.
- Zhang, C., Jian, Y., Ouyang, Z., and Vosoughi, S. Working memory identifies reasoning limits in language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 16896–16922, 2024.
- Zhang, J., Zhou, Z., Ji, X., Liu, S., and Zhao, Z. Capture: A benchmark and evaluation for lvlms in captcha resolving. *arXiv preprint arXiv:2512.11323*, 2025.
- Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

Appendix

A. More Details of Experimental Settings

We report Pass@1 under the evaluation protocol. Due to API budget constraints, we use a two-tier evaluation: we run Gemini3-Pro-High, Gemini3-Flash-High (Google, 2025), Qwen3-VL-Plus-ThinkingHigh (Bai et al., 2025), and Doubao-Seed-1.8-Thinking-HighEffort (ByteDance Seed, 2025) on the full testing set of 519 puzzles. We evaluate GPT-5.2-xHigh (OpenAI, 2025a) and Claude-Opus4.5-Extended-ThinkingHigh (Anthropic, 2025) on a 135-puzzle subset because max-reasoning closed-API inference exhibits substantial latency variance and occasional severe queuing delays at the step level, which makes running the full 519-puzzle suite prohibitively time and cost intensive (e.g, GPT-5.2-xHigh sometimes spends up to 1 hour to run 1 step). Unless stated otherwise, these are the default settings for all main results.

For GUI Agent framework, we evaluate all models using Browser-Use (browser-use, 2026) as the default agent framework for all main results and non-framework ablations, running each CAPTCHA as an interactive episode in a real browser. At each step t , the agent observes the current page state via $O_t = (I_t, D_t, \text{meta}_t)$, including a viewport screenshot, a structured DOM-derived view of interactive elements (indexed for interaction), and lightweight context such as the current URL and task information. The model then returns a JSON response that specifies the next browser actions, which are executed to advance the episode. We run the browser through Playwright (Microsoft, 2026) in visible (non-headless) mode and reset the agent state for every puzzle. Success or failure is recorded using the platform’s built-in outcome signals.

We additionally evaluate alternative orchestration frameworks including CrewAI (crewAI, 2026) and Claude Cowork (Anthropic, 2026) under the standard protocol.

B. Human Usability Evaluations

We report human usability results from two participant groups.

The Human column in Table 2 reports results from the CS-background human evaluation. This evaluation uses 10 puzzle instances for most CAPTCHA families and 8 instances for ShadowPlausible, giving 268 human trials in total. Participants were regular computer users. No explicit time limit was imposed, and participants submitted once they were confident. This evaluation achieves 98.8% Pass@1. The average completion time is 31.1 seconds per puzzle. The higher-time families are 3D_Viewpoint (120.8 seconds), Spooky_Jigsaw (71.8 seconds), Subway_Paths (71.4 seconds), and Multi_Script (63.1 seconds); the remaining task families have average completion times below 60 seconds.

The second evaluation covers participants without CS backgrounds. It includes 8 participants, each solving one instance from each of the 27 families, yielding 216 trials in total. This evaluation achieves 85.2% Pass@1. Table 4 summarizes the participant attributes and per-participant Pass@1. Together, the two evaluations report human performance for both CS/AI-background and non-CS/AI-background participant groups.

Attribute	P1	P2	P3	P4	P5	P6	P7	P8	Overall
Education	Master	Bachelor	Primary School	Bachelor	Master	PhD	Bachelor	Primary School	–
Age group	20–30	20–30	10–20	80–90	50–60	50–60	20–30	10–20	–
Expertise	Finance	Psychology	N/A	Business	Literature	Finance	Media	N/A	–
Pass@1 (%)	81.5	92.6	88.9	66.7	92.6	85.2	85.2	88.9	85.2

Table 4. **Non-CS/AI-background human usability evaluation.** Each participant attempted one instance from each of the 27 CAPTCHA families; Overall reports aggregate Pass@1 across 216 trials.

C. Robustness to Family-Aware Hint Adaptation

We evaluate a family-aware hint adaptation setting on the full 519-puzzle test set using Gemini-3-Flash-High with Browser-Use. Each CAPTCHA family is paired with one solving strategy hint generated by Gemini-3-Pro-High from the corresponding generator logic, frontend interaction code, server-side verification logic, and browser-agent task description. The hint describes observable webpage cues and potentially useful interactions for the family.

Under this family-aware hint adaptation attack, overall Pass@1 increases from 3.2% to 6.4%. The gain is concentrated in a small number of families, especially Red_Dot, while most CAPTCHA families remain at or near their baseline accuracy.

Overall, the Next-Gen CAPTCHA families remain robust under this hint-based adaptation setting.

Puzzle type	Baseline	Hint	Δ pp
Overall	3.2	6.4	+3.2
3D.Viewpoint	0.0	0.0	0.0
Backmost.Layer	5.0	0.0	-5.0
Box.Folding	10.0	5.0	-5.0
Color.Counting	0.0	5.0	+5.0
Dice.Roll.Path	15.0	20.0	+5.0
Dynamic.Jigsaw	0.0	0.0	0.0
Hole.Counting	0.0	5.0	+5.0
Illusory.Ribbons	0.0	0.0	0.0
Layered.Stack	0.0	0.0	0.0
Mirror	9.0	0.0	-9.0
Multi.Script	5.0	0.0	-5.0
Occluded.Pattern.Counting	0.0	5.0	+5.0
Red.Dot	15.0	100.0	+85.0
Rotation.Match	0.0	0.0	0.0
Shadow.Direction	0.0	0.0	0.0
Shadow.Plausible	0.0	12.5	+12.5
Spooky.Circle	20.0	5.0	-15.0
Spooky.Circle.Grid	0.0	0.0	0.0
Spooky.Jigsaw	0.0	0.0	0.0
Spooky.Shape.Grid	5.0	0.0	-5.0
Spooky.Size	5.0	15.0	+10.0
Spooky.Text	0.0	0.0	0.0
Static.Jigsaw	0.0	0.0	0.0
Structure.From.Motion	0.0	0.0	0.0
Subway.Paths	0.0	0.0	0.0
Temporal.Object.Continuity	0.0	0.0	0.0
Trajectory.Recovery	0.0	0.0	0.0

Table 5. **Robustness to family-aware hint adaptation.** Pass@1 (%) on the full 519-puzzle test set using Gemini-3-Flash-High with Browser-Use.

D. Dynamic-Family Payload Measurements

We measure the cold-load visual asset payload for CAPTCHA families that use GIF-based or other non-static visual assets. These measurements characterize transferred asset size under a cold-load setting and provide deployment-relevant information for selecting task families across network and device conditions. Table 6 summarizes the measured payloads.

In deployment, the system can choose task families based on connection quality and device capability, cache or preload assets when appropriate, and serve static or lower-bandwidth families for settings where lighter assets are preferred. These options allow deployments to balance task diversity, user experience, and system constraints while retaining the same rule-based verification interface.

E. Details of Next-Gen CAPTCHAs

This appendix provides implementation and evaluation details for our Next-Gen CAPTCHA families. The design goal is to move beyond *static, decomposable logic puzzles* and instead elicit failures that stem from the *Cognitive Gap*: robust interactive grounding, manipulation planning, and correct execution of action primitives under realistic UI constraints.

Design principles. Across families, we follow a shared set of principles:

- **Interactive grounding over static recognition.** Most tasks cannot be solved from a single screenshot plus short reasoning. Correct completion requires discovering actionable UI affordances and grounding targets throughout interaction.

Task family	Avg. cold-load payload (MiB)
Trajectory_Recovery	0.47
Temporal_Object_Continuity	0.54
Dynamic_Jigsaw	0.73
Spooky_Circle_Grid	3.40
Spooky_Shape_Grid	3.48
Spooky_Text	3.67
Spooky_Circle	3.86
Spooky_Size	6.77
Structure_From_Motion	7.73
Spooky_Jigsaw	9.23

Table 6. **Cold-load payloads for GIF-based dynamic families.** Payload is reported as the average amount of visual asset data loaded per puzzle instance under a cold-load setting.

Name	Brief description	Generative	Answer type	Benchmark size (#)	Targeted gaps
Mirror	Find mirror options that do <i>not</i> match the reflected reference	No	Select	11 [5]	G1
Dice_Roll_Path	Roll a die along a shown path; report the final top face	Yes	Numeric	20 [5]	G3, G4, G5
Shadow_Plausible	Pick images with physically plausible shadows in a grid	No	Select	8 [5]	G1
Layered_Stack	Select cells where top shape and counts in lower layers meet a rule	Yes	Select	20 [5]	G1, G3
Red_Dot	Timed clicks on appearing red dots until hit quota	Yes	Click position	20 [5]	G5
Color_Counting	Select the cells that meet the rule about the number of colours in the cell	Yes	Select	20 [5]	G3
Spooky_Circle	Count circles only visible via motion-contrast noise	Yes	Numeric	20 [5]	G2
Spooky_Size	Click largest/smallest target shape visible only via motion contrast	Yes	Click position	20 [5]	G2, G5
Box_Folding	Choose the folded cube that matches the given net	Yes	Select	20 [5]	G1, G4
3D_Viewpoint	Select all views showing the same colored-edge wireframe	Yes	Select	20 [5]	G1, G4, G5
Backmost_Layer	Click cells where the backmost (occluded) shape matches reference	Yes	Select	20 [5]	G1
Dynamic_Jigsaw	Drag/drop animated GIF pieces to complete a 3x3 jigsaw	Yes	Drag-and-drop	20 [5]	G2, G4, G5
Hole_Counting	Count topological holes in presented glyphs/shapes	Yes	Numeric	20 [5]	G1, G3
Illusory_Ribbons	Select cells containing exactly the target number of ribbon loops	Yes	Select	20 [5]	G1, G3
Occluded_Pattern_Counting	Count two specified shapes under a semi-transparent occluder	Yes	Numeric	20 [5]	G1, G3
Rotation_Match	Select tiles of the most frequent shape, ignoring rotation/color/texture	Yes	Select	20 [5]	G1, G4
Shadow_Direction	Match light-source direction from photorealistic shadows	Yes	Select	20 [5]	G1
Spooky_Circle_Grid	Count how many grid cells contain motion-contrast circles	Yes	Numeric	20 [5]	G2, G3
Spooky_Jigsaw	Drag/drop motion-contrast pieces to complete the jigsaw	Yes	Drag-and-drop	20 [5]	G2, G4, G5
Spooky_Shape_Grid	Select spooky cells with the target shape and rotation direction	Yes	Select	20 [5]	G2
Spooky_Text	Read and type text visible only via motion contrast	Yes	Text entry	20 [5]	G2
Static_Jigsaw	Drag/drop static pieces to complete a jigsaw	Yes	Drag-and-drop	20 [5]	G4, G5
Structure_From_Motion	Select GIF cells whose dot motion reflects the same rigid 3D shape	Yes	Select	20 [5]	G2
Trajectory_Recovery	Watch a reference trajectory GIF; select matching trajectory plots	Yes	Select	20 [5]	G2, G4
Multi_Script	Select cells containing any target characters (multi-script, transformed)	Yes	Select	20 [5]	G1
Subway_Paths	Select maps with the specified count of valid routes under stamp rules	Yes	Select	20 [5]	G3, G4
Temporal_Object_Continuity	Select GIF cells where identity changes behind occluders	Yes	Select	20 [5]	G2, G4
Summary	Captcha family from our GUI-Agent Defense System (27 types)	Yes	Mixed	519 [135]	G1–G5

Table 7. **Overview of Next-Gen CAPTCHA families.** Benchmark size is reported as **main [lite]** (lite: 5 instances per family). “Generative” denotes programmatic instance generation by our code system; “Targeted gaps” refers to the primary gaps in Section 4.

- **Action-primitive dependence.** Success requires the correct primitive (e.g., drag-and-drop, long-press, multi-step selection, constrained ordering). Incorrect primitives lead to hard failure even with strong perception.
- **Non-local dependence.** Early actions constrain later feasibility (e.g., piece placement affects future options), reducing the effectiveness of myopic step-by-step workflows.
- **Feedback that is informative but non-leaking.** UI feedback supports legitimate users (e.g., highlighting valid drop zones) without collapsing the task into a trivial reward signal exploitable by brute-force agents.
- **Human-feasible time/effort.** Each instance is designed to be solvable by typical users quickly, while remaining brittle for agents that mis-ground actions or affordances.

Instance generation and diversity. Each family instantiates challenges by sampling from a parameterized generator:

- **Content parameters** (visual themes, object sets, layouts) to prevent memorization and template matching.
- **Interaction parameters** (number of manipulable elements, allowed moves, constraints) to control difficulty.

- **Anti-shortcut constraints** that eliminate single-step solutions (e.g., enforcing multi-stage manipulation, requiring ordering constraints, or gating completion behind a correctly executed primitive).

We recommend maintaining a large instance pool with per-request randomization, and rotating asset sets periodically to reduce overfitting.

UI instrumentation and logging. For each attempt, we log a trajectory consisting of:

- **Observations:** screenshots (or DOM render snapshots) at each step.
- **Actions:** primitive type (click/drag/scroll/keyboard), coordinates/targets, and timestamps.
- **Agent deliberation signals:** step count, duration, and any available token-related signals for model-based agents.

These logs support the correlation analyses in Fig. 4 and enable failure-mode categorization (e.g., wrong primitive, mis-grounded target, premature submission).

Success criteria and verification. Each Next-Gen family defines a crisp completion predicate that is verified server-side to prevent client-side spoofing. Examples include:

- **State-based verification:** the final arrangement satisfies a constraint system (e.g., correct assembly/ordering).
- **Action-consistency checks:** required primitives occurred with valid targets (e.g., a drag sequence that ends in a valid drop zone).
- **Anti-replay:** per-instance nonces and short TTLs to prevent reuse of solutions.

Evaluation protocol. We evaluate Next-Gen families with browser-based agents under a consistent interaction budget:

- **Budgets:** maximum steps and wall-clock duration per attempt.
- **Constraints:** no privileged access beyond what a normal user/automation script can observe (i.e., no direct DOM oracle for hidden labels).
- **Metrics:** Pass@1 as the primary metric, plus trajectory statistics (Steps, Duration, and reasoning-related signals when available) used in Fig. 4.

Security and usability considerations. Next-Gen CAPTCHAs are intended to be:

- **Agent-defensive:** brittle to incorrect primitives and shallow workflow decomposition.
- **User-friendly:** solvable with intuitive affordances and minimal cognitive load.
- **Robust in deployment:** server-side verification, rate limiting, and anomaly detection over trajectories to reduce large-scale automated attacks.

E.1. Next-Gen CAPTCHA Examples Gallery

The following gallery presents representative instances from our Next-Gen CAPTCHA families. Each example is intended to illustrate *why* the family remains difficult for current GUI agents even when static perception is strong.



Figure 9. Five concrete examples of Next-Gen CAPTCHAs. Dynamic Jigsaw, Mirror, Illusory Ribbons, Layer Stack and Multi Script.

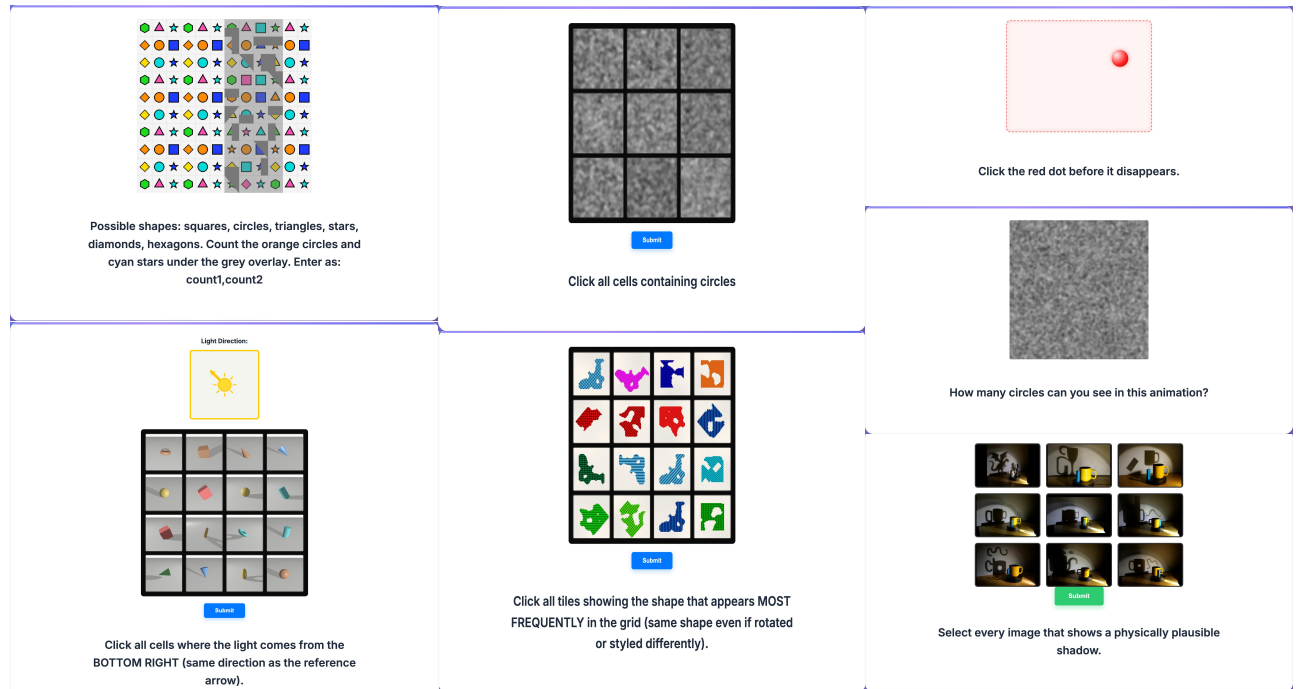


Figure 10. Seven concrete examples of Next-Gen CAPTCHAs. Occluded Pattern Counting, Shadow Direction, Spooky Circle Grid, Rotation Match, Red Dot, Spooky Circle and Shadow Plausible.

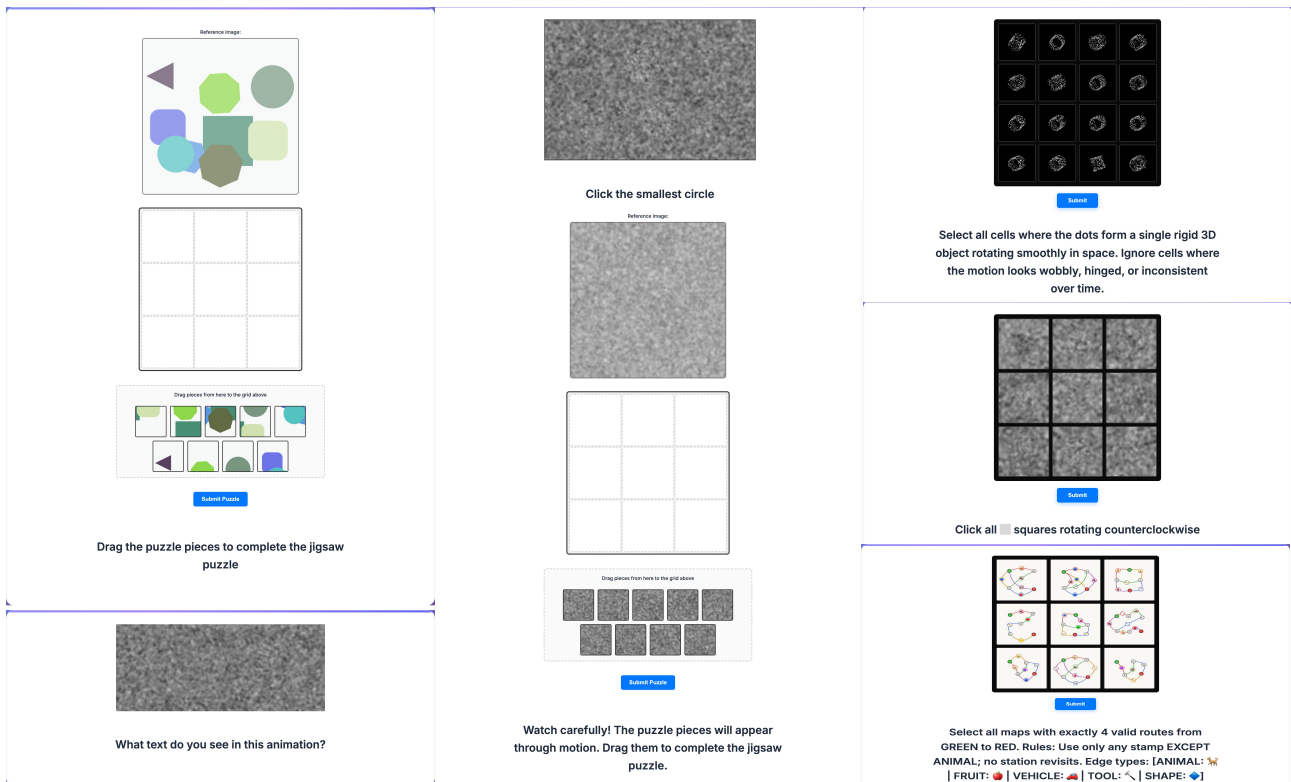


Figure 11. Seven concrete examples of Next-Gen CAPTCHAs. Static Jigsaw, Spooky Text, Spooky Size, Spooky Jigsaw, Structure From Motion, Spooky Shape Grid and Subway Path.

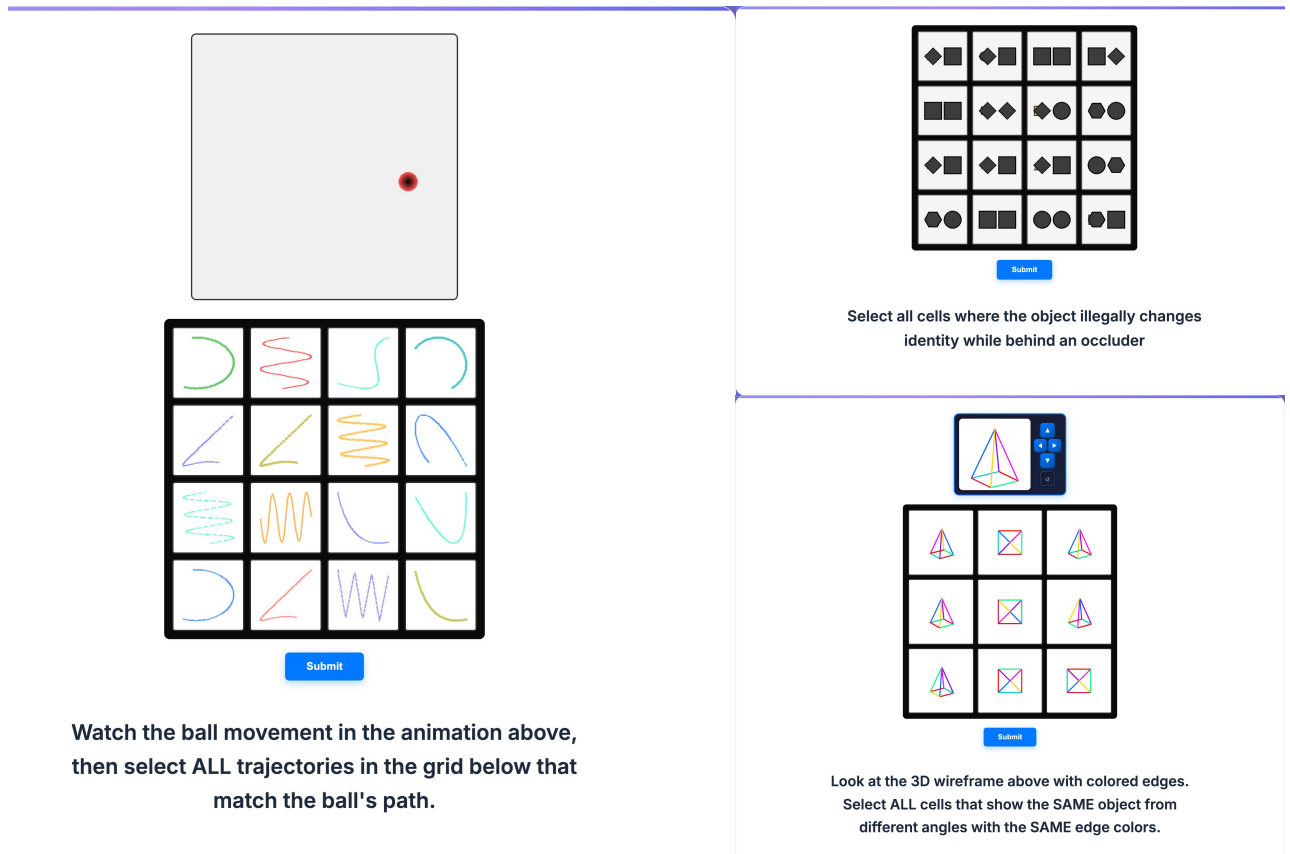


Figure 12. Three concrete examples of Next-Gen CAPTCHAs. Trajectory Recovery, Temporal Object Continuity and 3D Viewpoint.